Karpenko Bohdan AIS ID: 135007

Data structures and algorithms

Writing and testing the effectiveness of algorithms:
Hash table with open addressing,
Red-Black tree,
2-3 tree

Karpenko Bohdan AIS ID: 135007

Принцип тестування

Тестування виконувалось на послідовності даних яка складалась з: n вставок, n пошуків, n видаленнь. Де n кількість операцій що буде зазначатись в подальших таблицях та графіках.

Для тесту було використано розмір n що становив 10, 100, 1000, 10,000, 100,000, 1,000,000, 10,000,000.

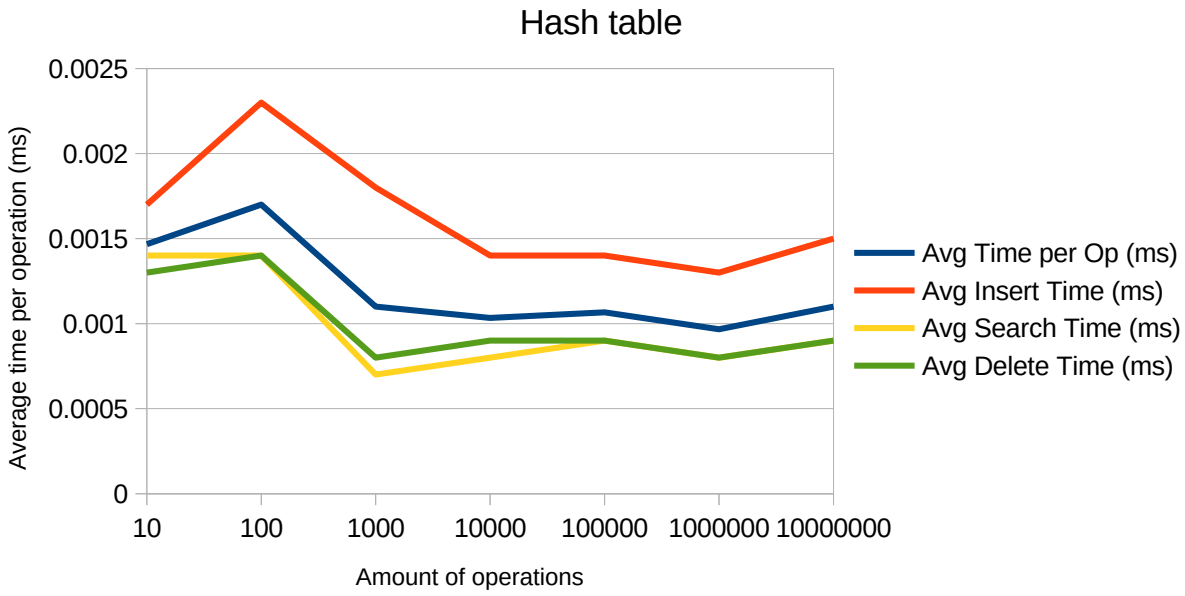
Під час тестування записувалась швидкість виконання кожної операції (вставки, пошуку та видалення) та максимальний об’єм використаної пам’яті реалізацією алгоритму.

Karpenko Bohdan AIS ID: 135007

Results of performance speed testing.

1. Hash table

Amount of operations	Avg Time per Op (ms)	Avg Insert Time (ms)	Avg Search Time (ms)	Avg Delete Time (ms)
10	0.00047	0.0017	0.0014	0.0013
100	0.00047	0.0023	0.0014	0.0014
1000	0.00023	0.0018	0.0007	0.0008
10000	0.00027	0.0014	0.0008	0.0009
100000	0.00030	0.0014	0.0009	0.0009
1000000	0.00027	0.0013	0.0008	0.0008
10000000	0.00030	0.0015	0.0009	0.0009

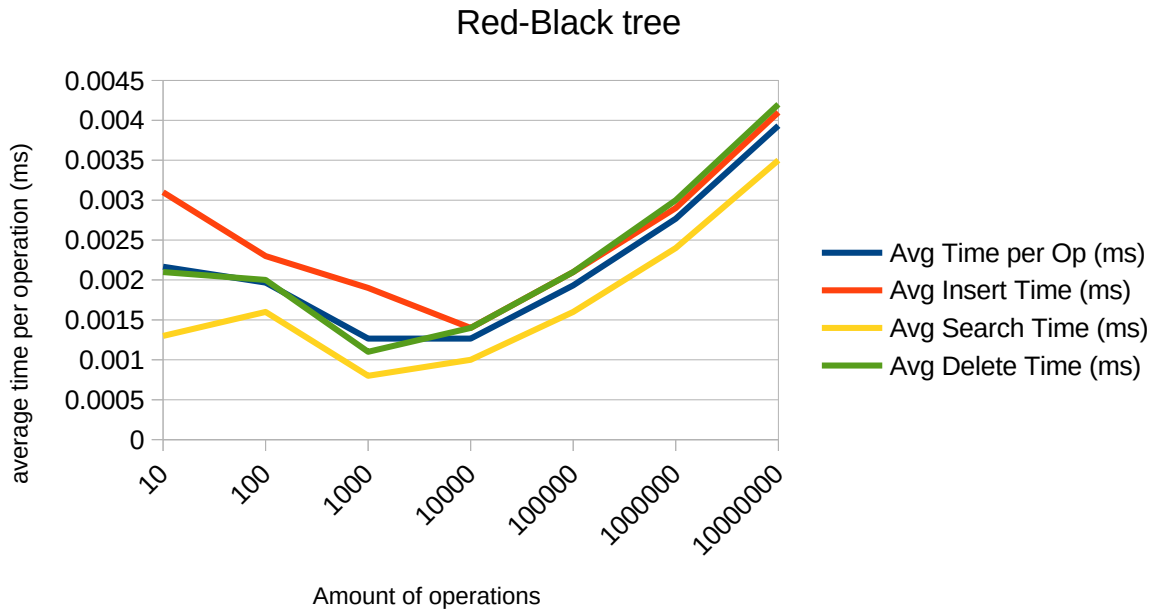


Як видно з графіку із 1000 елементів швидкість виконання операцій стає сталою. Це відбувається через те, що збільшення розмірів таблиці відбувається за правилом:
newSize = oldSize*2;
Що призводить до частих збільшень розмірів таблиці на початку її заповнення, але з часом кількість розширень таблиці на вставку буде падати, що підтверджує константну складність. Цим же правилом і пояснюється більша кількість часу на операцію що спостурігалась на початку заповнення таблиці. Трохи більший час необхідний на вставку пояснюється тим, що саме в час цієї операції враховуєтсья зміна розміру таблиці, що робить її трохи повільнішою за видалення та пошук.

Karpenko Bohdan AIS ID: 135007

2. Red-Black tree

Amount of operations	Avg Time per Op (ms)	Avg Insert Time (ms)	Avg Search Time (ms)	Avg Delete Time (ms)
10	0.00217	0.0031	0.0013	0.0021
100	0.00197	0.0023	0.0016	0.002
1000	0.00127	0.0019	0.0008	0.0011
10000	0.00127	0.0014	0.001	0.0014
100000	0.00193	0.0021	0.0016	0.0021
1000000	0.00277	0.0029	0.0024	0.003
10000000	0.00393	0.0041	0.0035	0.0042



Як видно з графіка, при малих об’ємах даних (до 100 елементів) спостерігаються значні коливання у середньому часі виконання операцій. Це зумовлено не лише ефектами кешування в процесорі, а й похибками вимірювання, що виникають через особливості роботи операційної системи, планувальника потоків та інших фонових процесів, які мають більший вплив при низькій кількості ітерацій.

Починаючи з приблизно 10 000 операцій, час виконання стабілізується та демонструє поступове зростання. Це узгоджується з теоретичною логарифмічною складністю червоно-чорного дерева (O(log n)), оскільки зі збільшенням кількості елементів структура дерева ускладнюється, а кількість необхідних ротацій і змін кольорів для підтримання балансу зростає.

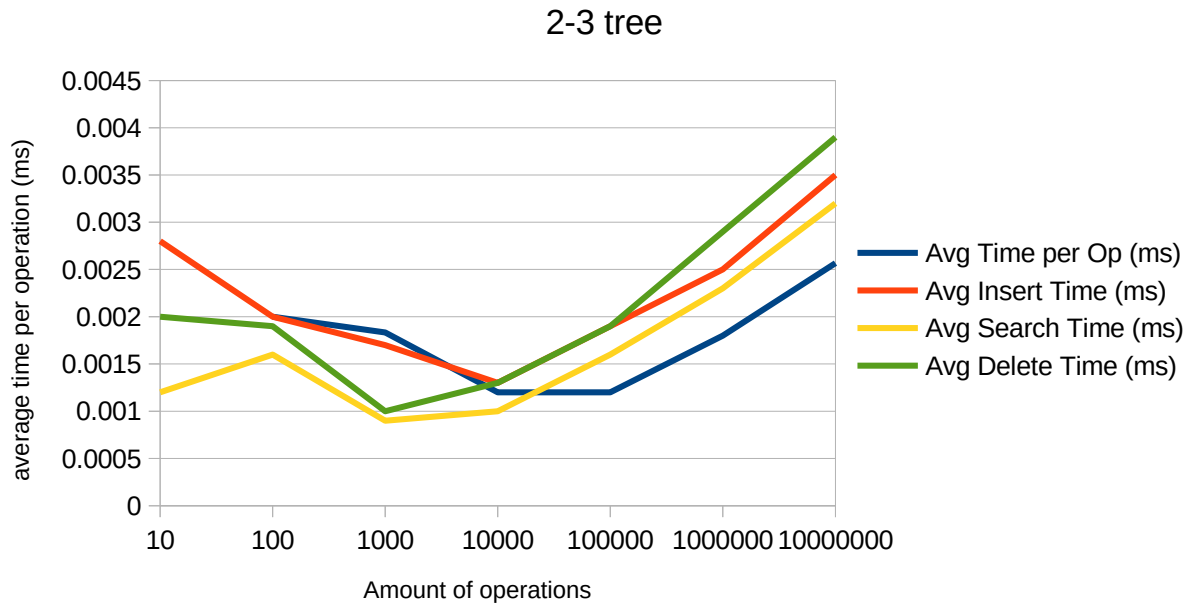
Найбільше часу витрачається на вставку та видалення, що пояснюється потребою у балансуванні дерева після кожної з цих операцій. Вони вимагають додаткових змін у структурі вузлів, щоб зберегти властивості червоно-чорного дерева. Натомість пошук є найшвидшою операцією, оскільки не змінює структуру дерева, а лише проходить по вже впорядкованим вузлам.

Таким чином, отримані результати підтверджують передбачувану логарифмічну поведінку червоно-чорного дерева при збільшенні обсягу даних, а також демонструють відносну стабільність та ефективність пошуку порівняно з іншими операціями.

Karpenko Bohdan AIS ID: 135007

3. 2-3 tree

Amount of operations	Avg Time per Op (ms)	Avg Insert Time (ms)	Avg Search Time (ms)	Avg Delete Time (ms)
10	0.00200	0.0028	0.0012	0.002
100	0.00183	0.002	0.0016	0.0019
1000	0.00120	0.0017	0.0009	0.001
10000	0.00120	0.0013	0.001	0.0013
100000	0.00180	0.0019	0.0016	0.0019
1000000	0.00257	0.0025	0.0023	0.0029
10000000	0.00353	0.0035	0.0032	0.0039



На перших етапах (до 100 елементів) середній час виконання операцій у 2-3 дереві демонструє значні коливання. Це пояснюється не лише ефектами кешування, коли вся структура вміщується в кеш процесора, а й похибками вимірювання, що є типовими для малих обсягів операцій. Такі похибки виникають через фонову активність операційної системи, планування потоків і інші зовнішні фактори, які мають непропорційно великий вплив за низької кількості ітерацій.

Після 1 000–10 000 операцій час виконання стабілізується, а далі поступово зростає. Це пов’язано з логарифмічною складністю основних операцій у 2-3 дереві, яке підтримує баланс за допомогою розщеплення або об'єднання вузлів. Зі зростанням обсягу даних ці допоміжні дії стають дедалі частішими, що й обумовлює ріст часу.

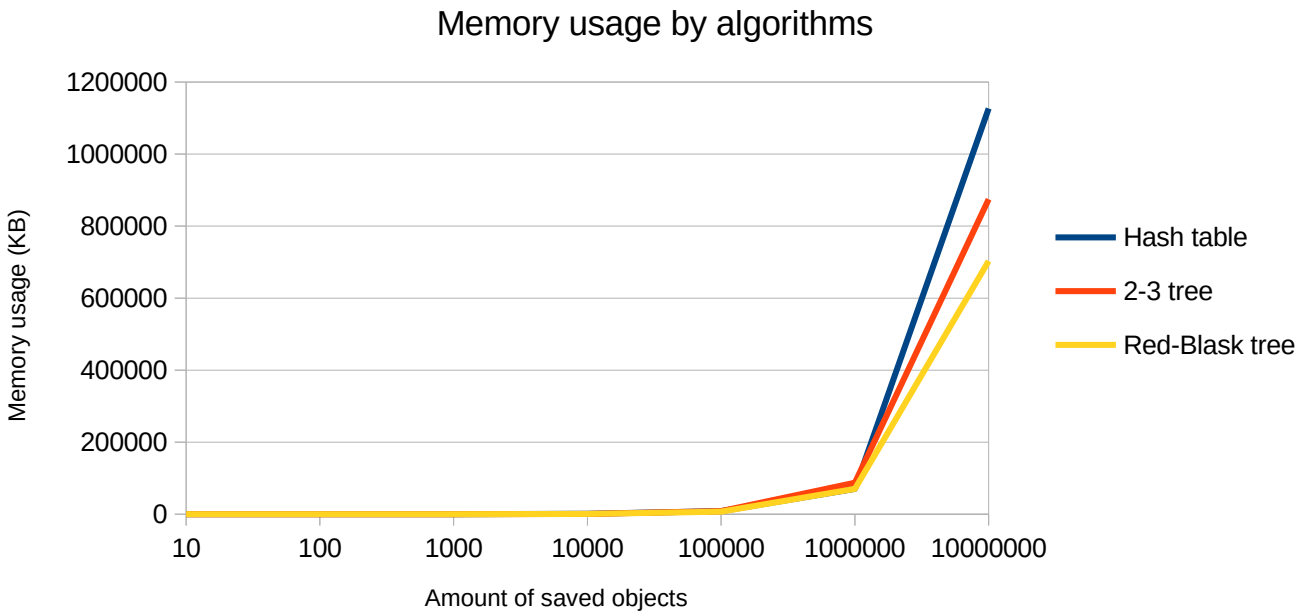
Як видно з графіка, найбільше часу потребують операції вставки та видалення, адже вони включають додаткові процедури перебудови дерева для збереження його збалансованої структури. В особливості, видалення потребує як пошуку вузла, так і потенційного злиття або переміщення ключів між вузлами, що робить його ресурсоємним. Натомість пошук виконується найшвидше, оскільки є найбільш прямолінійною операцією — воно не змінює структуру дерева, а лише проходить по впорядкованих вузлах.

Загалом, отримані результати підтверджують очікувану асимптотику часу виконання операцій у 2-3 дереві та демонструють його ефективність при роботі з великими обсягами даних.

Karpenko Bohdan AIS ID: 135007

Порівняння споживання пам’яті алгоритмами

number of insertions	Hash table memory usage (KB)	2-3 tree memory usage (KB)	Red-Blask tree memory usage (KB)
10	2	0	0
100	8	7	7
1000	68	88	70
10000	1100	875	703
100000	8800	8741	7031
1000000	70400	87419	70312
10000000	1126400	874447	703125



Згідно з представленими даними, алгоритми демонструють різну ефективність у використанні оперативної пам’яті при зростанні кількості вставок. Найбільше споживання пам’яті спостерігається у хеш-таблиці. Це пояснюється її структурною особливістю: задля забезпечення постійного часу доступу, хеш-таблиця зазвичай підтримує резервну кількість комірок, що перевищує кількість фактичних елементів. Частина цієї пам’яті залишається вільною для уникнення колізій та забезпечення швидкої вставки й пошуку. Таким чином, навіть за однакової кількості даних, хеш-таблиця займає більше простору, ніж дерева.

Щодо деревоподібних структур, 2-3 дерево виявилось менш ефективним у плані споживання пам’яті порівняно з червоно-чорним деревом. Це пов’язано з

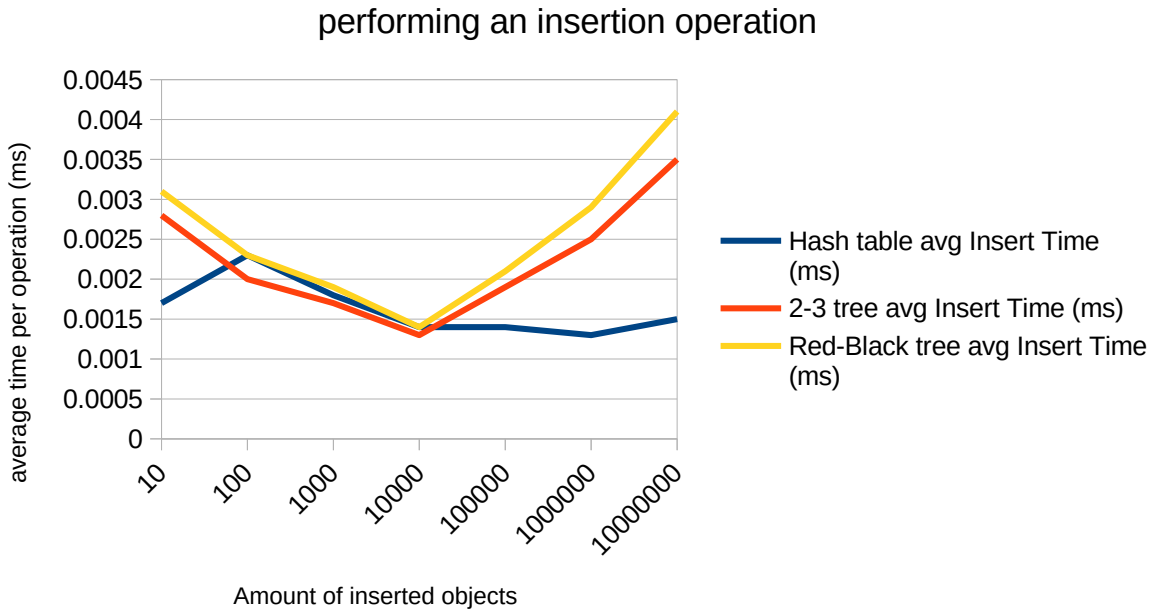
особливостями реалізації: кожен вузол 2-3 дерева містить дві повноцінні комірки даних, а не лише вказівники на них. Такий підхід був обраний з міркувань оптимізації: зменшення кількості викликів `malloc()` та, відповідно, системних звернень дозволяє підвищити загальну швидкість операцій над деревом, зокрема на етапах великої кількості вставок.

У свою чергу, червоно-чорне дерево зберігає лише одну комірку даних на вузол, що робить його найекономнішим з точки зору споживання пам’яті серед представлених структур. Незважаючи на додаткову інформацію про колір вузла та посилання на батьківську ноду, ця реалізація залишається компактною та пам’яттєво ефективною.

Karpenko Bohdan AIS ID: 135007

Порівнняння швидкості виконання операції вставки.

number of insertions	Hash table avg Insert Time (ms)	2-3 tree avg Insert Time (ms)	Red-Black tree avg Insert Time (ms)
10	0.0017	0.0028	0.0031
100	0.0023	0.002	0.0023
1000	0.0018	0.0017	0.0019
10000	0.0014	0.0013	0.0014
100000	0.0014	0.0019	0.0021
1000000	0.0013	0.0025	0.0029
10000000	0.0015	0.0035	0.0041



Щодо деревоподібних структур, 2-3 дерево показує кращі результати за середнім часом вставки у порівнянні з червоно-чорним деревом. Це пояснюється тим, що в 2-3 дереві пошук відповідного місця для вставки зазвичай є швидшим, а сама вставка не завжди потребує створення нового вузла. У тих випадках, коли ключ може бути розміщений у другій позиції вже існуючого вузла, у поточній реалізації використовується вже виділена пам’ять, що дозволяє уникнути виклику `malloc()`, тим самим зменшуючи загальний час операції.

Червоно-чорне дерево, хоча й має логарифмічну складність, вимагає обов’язкової перевірки балансу дерева після кожної вставки та, у багатьох випадках, виконання ротацій і зміни кольору вузлів, що ускладнює й уповільнює цю операцію порівняно з 2-3 деревом.

На малих об’ємах даних (до 100 операцій) спостерігаються типові коливання у часі виконання. Вони, як і раніше, пояснюються впливом зовнішніх факторів, таких як кешування, фонові системні процеси, планувальник операційної системи, а також недостатньо великою кількістю ітерацій для усереднення результатів.

Таким чином, аналіз показує, що хеш-таблиця забезпечує найшвидшу вставку, 2-3 дерево — оптимальний компроміс між складністю структури та економією ресурсів, тоді як червоно-чорне дерево поступається через вищу вартість балансування при вставці.

Щодо деревоподібних структур, 2-3 дерево показує кращі результати за середнім часом вставки у порівнянні з червоно-чорним деревом. Це пояснюється тим, що в 2-3 дереві пошук відповідного місця для вставки зазвичай є швидшим, а сама вставка не завжди потребує створення нового вузла. У тих випадках, коли ключ може бути розміщений у другій позиції вже існуючого вузла, у поточній реалізації використовується вже виділена пам’ять, що дозволяє уникнути виклику `malloc()`, тим самим зменшуючи загальний час операції.

Червоно-чорне дерево, хоча й має логарифмічну складність, вимагає обов’язкової перевірки балансу дерева після кожної вставки та, у багатьох випадках, виконання ротацій і зміни кольору вузлів, що ускладнює й уповільнює цю операцію порівняно з 2-3 деревом.

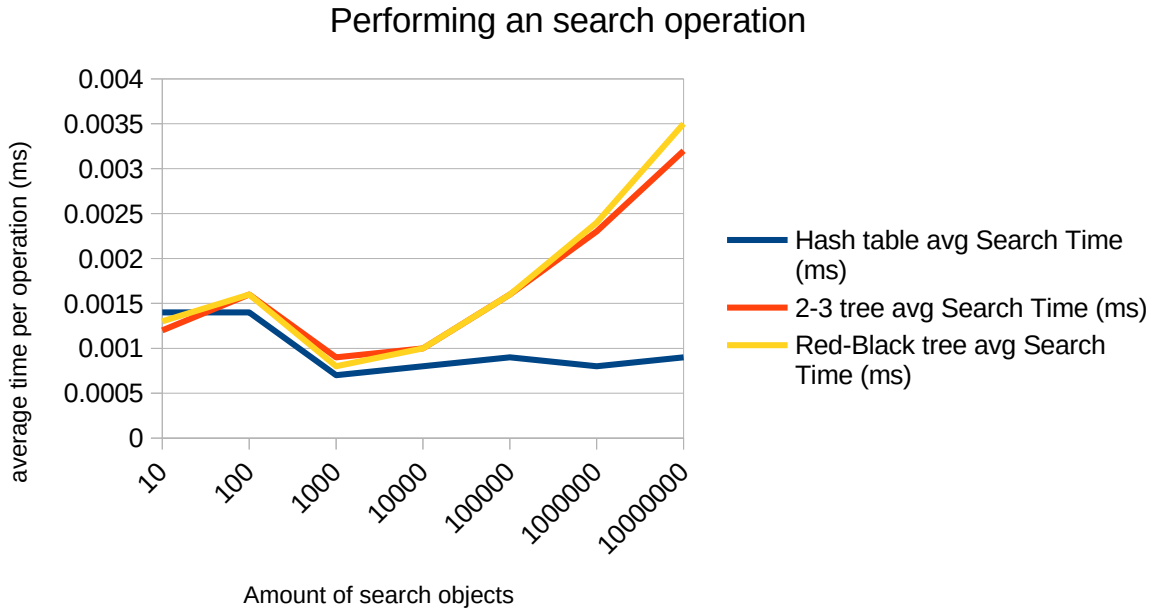
На малих об’ємах даних (до 100 операцій) спостерігаються типові коливання у часі виконання. Вони, як і раніше, пояснюються впливом зовнішніх факторів, таких як кешування, фонові системні процеси, планувальник операційної системи, а також недостатньо великою кількістю ітерацій для усереднення результатів.

Таким чином, аналіз показує, що хеш-таблиця забезпечує найшвидшу вставку, 2-3 дерево — оптимальний компроміс між складністю структури та економією ресурсів, тоді як червоно-чорне дерево поступається через вищу вартість балансування при вставці.

Karpenko Bohdan AIS ID: 135007

Порівнняння швидкості виконання операції пошуку.

number of searches	Hash table avg Search Time (ms)	2-3 tree avg Search Time (ms)	Red-Black tree avg Search Time (ms)
10	0.0014	0.0012	0.0013
100	0.0014	0.0016	0.0016
1000	0.0007	0.0009	0.0008
10000	0.0008	0.001	0.001
100000	0.0009	0.0016	0.0016
1000000	0.0008	0.0023	0.0024
10000000	0.0009	0.0032	0.0035



Як видно з результатів експерименту, найшвидший пошук забезпечує хеш-таблиця, що цілком очікувано. Завдяки своїй природі, хеш-таблиця дозволяє здійснювати доступ до елемента у сталий (амортизований) час, незалежно від розміру структури. Пошук здійснюється за ключем через обчислення хеш-функції, що не вимагає обходу дерева або перебалансування.

У випадку з 2-3 деревом та червоно-чорним деревом, час пошуку зростає із збільшенням кількості елементів. Це зумовлено тим, що пошук в обох структурах має логарифмічну складність O(log n), але на практиці кількість кроків у дереві зростає зі збільшенням його висоти. Особливо це помітно у 2-3 дерева, де, згідно з реалізацією, пошук може супроводжуватись додатковими перевірками

внутрішньої структури вузлів (дві або одна ключові комірки), що також впливає на загальний час.

Також слід відзначити, що червоно-чорне дерево показує кращі результати за 2-3 дерево. Це пояснюється меншою кількістю перевірок усередині вузла та простою структурою: кожен вузол містить лише один ключ, і, відповідно, пошук у ньому є прямолінійним.

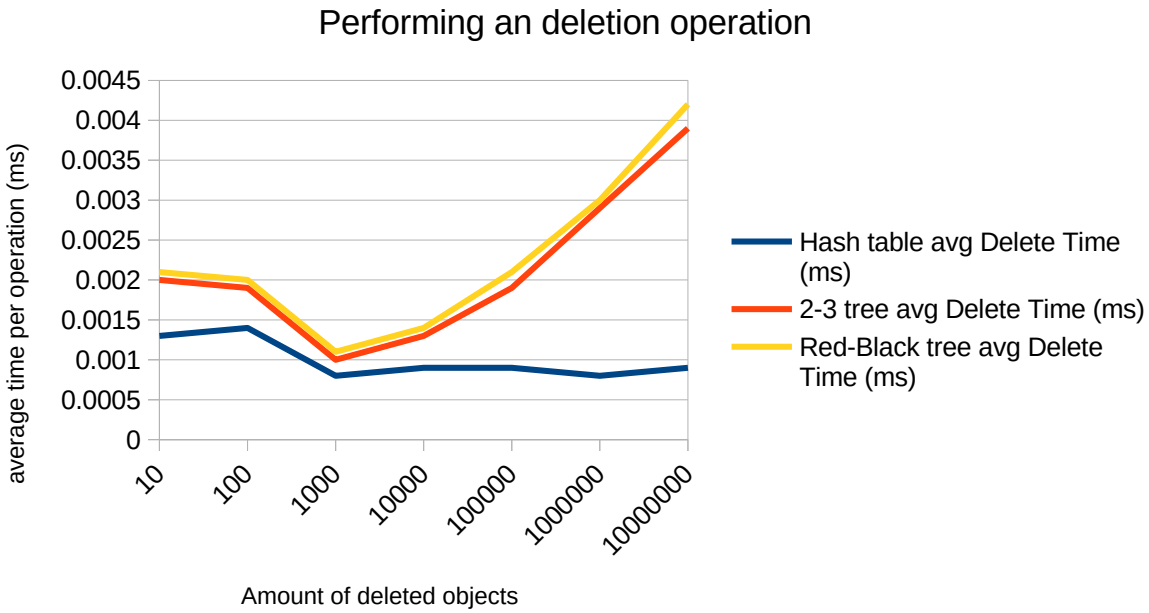
На малих обсягах (до 100 операцій) спостерігаються звичні флуктуації результатів, що, як і раніше, пояснюються похибками, спричиненими впливом кешування, активністю операційної системи та недостатньою статистичною вибіркою.

У підсумку, результати підтверджують очікувану поведінку структур: хеш-таблиця — найефективніша для пошуку, тоді як деревоподібні структури демонструють передбачуване логарифмічне зростання часу операції, причому червоно-чорне дерево є дещо швидшим за 2-3 дерево в цьому аспекті.

Karpenko Bohdan AIS ID: 135007

Порівнняння швидкості виконання операції видалення.

number of deletes	Hash table avg Delete Time (ms)	2-3 tree avg Delete Time (ms)	Red-Black tree avg Delete Time (ms)
10	0.0013	0.0020	0.0021
100	0.0014	0.0019	0.0020
1000	0.0008	0.0010	0.0011
10000	0.0009	0.0013	0.0014
100000	0.0009	0.0019	0.0021
1000000	0.0008	0.0029	0.0030
10000000	0.0009	0.0039	0.0042



Як демонструють результати експерименту, найменший середній час видалення забезпечує хеш-таблиця. Завдяки доступу до елементів за ключем, операція видалення в хеш-таблиці виконується без потреби проходження структури або перебудови її елементів. Сама по собі операція полягає у позначенні або очищенні відповідної комірки, що займає стабільний і мінімальний час навіть при великій кількості елементів.

У випадку з деревами, як 2-3 дерево, так і червоно-чорне дерево демонструють більший середній час видалення. Це зумовлено складністю логіки, необхідної для збереження балансу після видалення елемента. Обидві структури потребують

додаткових перевірок і перебалансування дерева, включно з ротаціями (для Red-Black) або злиттям вузлів (для 2-3 дерева), що суттєво збільшує час операції.

При цьому червоно-чорне дерево виявилось найповільнішим у цьому аспекті. Це пояснюється більш жорсткими умовами підтримки балансу (врахування кольору вузлів, потреба в додаткових ротаціях), які часто виникають саме після видалення. Натомість 2-3 дерево у певних випадках може уникнути глибоких змін, якщо видалення відбувається з вузла з двома ключами — тоді перерозподіл ключів або злиття можуть не знадобитися, що дещо зменшує час операції.

На початкових етапах (до 100 операцій) спостерігаються стандартні флуктуації, що, як і в попередніх випадках, спричинені зовнішніми чинниками: кешуванням, системними викликами, фоновими процесами та невеликою вибіркою для усереднення.

Загалом, результати підтверджують, що видалення в хеш-таблиці є найбільш ефективним, у той час як деревоподібні структури, хоч і зберігають баланс, вимагають додаткових дій після видалення, що підвищує середній час виконання цієї операції.

Karpenko Bohdan AIS ID: 135007

Підсумок

На основі проведеного аналізу можна зробити висновки щодо доцільності використання різних структур даних залежно від поставлених задач:

- Хеш-таблиця виявилась найбільш ефективною за швидкістю виконання всіх базових операцій (вставка, пошук, видалення), особливо на великих об’ємах даних. Це робить її оптимальним вибором для задач, де ключовими є швидкий доступ за унікальним ключем (наприклад, кешування, словники, реалізація асоціативних масивів). Проте варто враховувати її високе споживання пам’яті, що може бути критичним у системах з обмеженими ресурсами.
- 2-3 дерево демонструє добрий баланс між швидкодією та стабільністю, а також помірне використання пам’яті. Завдяки особливостям реалізації, де деякі вставки можуть відбуватись без створення нових вузлів, ця структура може бути ефективною у системах з високим навантаженням на вставку та частими модифікаціями, наприклад, у базах даних або файлових системах. Водночас його ефективність зменшується при великих об’ємах даних через зростання часу пошуку.
- Червоно-чорне дерево продемонструвало найкращу пам’яттєву ефективність серед досліджуваних структур, що робить його доречним вибором у випадках, де важливо зберігати великий обсяг впорядкованих даних з обмеженим обсягом пам’яті (наприклад, у компіляторах, індексації або реалізації збалансованих множин). Його логарифмічна складність всіх операцій забезпечує стабільну продуктивність, однак висока вартість операції видалення робить його менш привабливим для систем з частими модифікаціями.

Таким чином, вибір структури даних повинен базуватися на конкретних потребах: швидкість доступу, обмеження пам’яті, частота змін і необхідність упорядкування, оскільки кожна структура має свої сильні й слабкі сторони в залежності від контексту використання.